DB 학습을 앞두고
0. 들어가며
1. 왜 하필 지금, DB를 배우는가?
2. DB 학습은 두 단계로 진행됩니다
3. Day 1-2: SQL
3.1 SQL은 사서에게 보내는 표준 요청서
3.2 SQL은 4개의 동사로 이루어져 있다
3.3 "ORM 쓸 건데 SQL을 굳이?"
4. Day 3-6: 모델 관계, "서비스 설계의 지도"
4.1 거의 모든 서비스는 "관계"로 만들어진다
4.2 관계 설계의 비유: 인간 관계도
4.3 "저는 프론트엔드 지망인데 모델 관계까지?"
5. 학습 가이드
5.1 ERD를 먼저 그리고, 코드를 나중에
5.2 ORM이 만든 SQL을 직접 보기
5.3 마이그레이션은 여러번 할 생각으로 임하기
5.4 AI에게 SQL/모델을 시킬 때
6. 학습 로드맵에서의 위치

DB 학습을 앞두고

0. 들어가며

지금까지 우리는 **Django로 서비스의 뼈대를 세우는 법**을 익혔습니다. 그런데 한 가지, 지금까지 의식하지 않고 지나온 것이 있습니다. 바로 **데이터** 입니다.

`Article.objects.all()` 한 줄을 입력하면 게시글이 줄줄이 딸려 나옵니다. 하지만 **그 데이터는 어디에 어떤 모양으로 저장되어 있고, 어떤 언어로 꺼내지는 걸까요?**

> **사고의 전환 포인트**
>
> 코드는 자주 다시 쓰지만, **데이터는 좀처럼 죽지 않습니다.** 서비스가 망해도 데이터는 백업되어 살아남고, 프레임워크가 바뀌어도 DB는 그대로 유지되는 일이 흔합니다. 그래서 좋은 개발자일수록 **"가장 오래 살아남는 자산"**인 데이터의 구조를 더 진지하게 임합니다.

---

1. 왜 하필 지금, DB를 배우는가?

이미 Django Model을 만들어 봤고 ORM도 써봤습니다. 그런데 왜 지금 굳이 SQL을 따로 배워야 할까?

시점	알고 있던 것	가려져 있던 것
Django Model까지	"모델을 만들면 테이블이 생긴다"	그 테이블이 실제로 어떻게 생겼는지
ORM 사용까지	"`.filter()` 로 데이터를 가져온다"	그게 어떤 SQL로 번역되는지

시점	알고 있던 것	가려져 있던 것
DB 학습 이후	**양쪽을 모두 보고 직접 다룰 수 있음**	-

> **추상화는 그 아래를 본 사람에게만 진짜 의미가 생깁니다.**
>
> ORM은 SQL을 "쓰지 않게 해주는 도구"가 아니라, **"SQL을 빠르게 쓰게 해주는 도구"** 입니다. 통역사가 일을 잘하는지 평가하려면, 원어를 어느 정도는 알아야 합니다.

---

2. DB 학습은 두 단계로 진행됩니다

이번 6일은 의도적으로 두 덩어리로 나뉘어 있습니다.

```
[ Day 1-2 ] SQL        <== "DB와 직접 대화하기"
   |                        Django를 잠시 내려놓고, raw 데이터베이스를 맨손으로 만집니다.
   |                        SELECT / INSERT / UPDATE / DELETE / JOIN
   ▼
[ Day 3-6 ] Django 모델 관계    <== "관계로 서비스 설계하기"
                                    게시글-댓글, User-게시글, 좋아요, 팔로우,,,
                                    실제 서비스의 90%는 결국 "관계"로 만들어집니다.
```

이 두 덩어리는 **별개의 주제가 아니라 같은 산을 양쪽에서 오르는 길**입니다. 1-2일차에서 손으로 짠 SQL이, 3-6일차에서 ORM의 한 줄로 압축되는 경험을 직접 하게 됩니다.

---

3. Day 1-2: SQL

3.1 SQL은 사서에게 보내는 표준 요청서

> **"2000년 이후 출간된 과학 소설을, 제목과 저자만 최신순으로 보여주세요."**

SQL은 **데이터베이스라는 거대한 도서관의 사서(DBMS)에게 건네는, 약속된 양식의 요청서**입니다. 사람이 평소 쓰는 모호한 말이 아니라, 정해진 키워드와 순서로만 요청을 적습니다. 그 양식 덕분에 사서는 매번 정확히 같은 방식으로 답을 돌려줍니다.

3.2 SQL은 4개의 동사로 이루어져 있다

```
SQL 명령어 == 4가지 그룹
├── DDL  (Data Definition)     -- 책장 자체를 만들고 부순다
|     CREATE / ALTER / DROP
├── DML  (Data Manipulation)   -- 책을 꽂고, 바꾸고, 뺀다
|     INSERT / UPDATE / DELETE
├── DQL  (Data Query)          -- 책을 찾는다
|     SELECT
└── DCL  (Data Control)        -- 책장 열쇠를 관리한다
      GRANT / REVOKE / COMMIT / ROLLBACK
```

> 처음에는 SQL이 단어가 많아 보이지만, 결국 **"만들기 / 넣기 / 찾기 / 바꾸기 / 지우기"** 다섯 개로 모든 것이 표현됩니다.

3.3 "ORM 쓸 건데 SQL을 굳이?"

실무에서 SQL을 직접 들여다봐야 하는 순간은 의외로 많습니다.

상황	SQL을 알아야 하는 이유
성능 문제	ORM 한 줄이 뒤에서 100개의 쿼리를 날리고 있을 수 있음 (N+1 문제)
데이터 분석/추출	운영 중인 DB에서 통계를 뽑을 때, 보통 ORM 없이 SQL로 직접 작성
다른 팀/회사 코드	모든 백엔드가 Django는 아님. SQL은 어디서나 통하는 공용어

> **SQL은 "필요할 때" 쓰는 게 아니라, "뒤에서 항상 돌고 있는 것"** 입니다. 보이지 않는다고 없는 것이 아닙니다.

---

4. Day 3-6: 모델 관계, "서비스 설계의 지도"

4.1 거의 모든 서비스는 "관계"로 만들어진다

우리가 자주 쓰는 서비스를 떠올려 봅시다.

- **인스타그램**: 유저(1) - 게시글(N) - 댓글(N), 유저(M) - 좋아요(N), 유저(M) - 팔로우(M)
- **유튜브**: 채널(1) - 영상(N), 영상(1) - 댓글(N), 유저(M) - 구독(M)
- **배달의민족**: 가게(1) - 메뉴(N), 유저(1) - 주문(N), 유저(M) - 찜(M)

서비스의 기능들은 결국 **"누가 무엇과 어떤 식으로 연결되어 있는가"** 라는 단 하나의 질문으로 환원됩니다. 그래서 4일을 들여 다음 두 가지 패턴만 확실하게 익힙니다.

```
[ N:1 (Many to One) ]  여러 자식이 하나의 부모를 가리킴
   - 게시글(N) -> 작성자(1)
   - 댓글(N)   -> 게시글(1)
   - 댓글(N)   -> 작성자(1)
   => Django: ForeignKey

[ N:M (Many to Many) ] 양쪽 모두 여러 개를 가리킬 수 있음
   - 유저(N) <=> 좋아요한 게시글(M)
   - 유저(N) <=> 팔로우한 유저(M)   ※ 자기 자신과의 관계
   => Django: ManyToManyField
```

4.2 관계 설계의 비유: 인간 관계도

> **N:1 (한 명의 부모 - 여러 자식)**: 댓글에게 "너 어느 글에 달린 댓글이야?" 라고 물으면 **딱 하나**의 답이 나와야 합니다. 반대로 게시글에게 "너한테 달린 댓글 다 줘" 라고 하면 **여러 개**가 나옵니다.
>
> **N:M (좋아요, 팔로우)**: 한 사람이 여러 글에 좋아요를 누를 수 있고, 한 글도 여러 사람에게 좋아요를 받을 수 있습니다. 이렇게 양쪽이 모두 "여러"인 관계는 **중간에 다리(중개 테이블)** 가 필요합니다.

4.3 "저는 프론트엔드 지망인데 모델 관계까지?"

화면은 결국 **데이터를 보여주는 창** 입니다. 화면에 보이는 모든 카드, 리스트, 댓글, 알림은 누군가가 설계한 모델 관계의 결과물입니다.

```
[ Vue 컴포넌트 ]
      |          "이 카드에 작성자 이름이랑 좋아요 수 같이 보여줘"
      ▼
[ API 응답 형태 ]   <== 이 형태는 백엔드의 모델 관계가 결정함
      |
      ▼
[ Django Model 관계 ]
```

> 모델 관계를 모르면, "왜 이 정보는 한 번의 API 호출로 못 받죠?" 라는 질문에 답하지 못합니다. 모델 관계를 아는 프론트엔드 개발자는 백엔드와 **같은 언어로 대화하는 사람**이 됩니다.

---

5. 학습 가이드

5.1 ERD를 먼저 그리고, 코드를 나중에

복잡한 모델 관계는 머릿속에서 정리되지 않습니다. 종이든 화이트보드든 **"네모(테이블) 사이에 선(관계) 긋기"** 부터 시작하세요. 코드는 그 다음입니다.

5.2 ORM이 만든 SQL을 직접 보기

`Database Client` 같은 DB 뷰어로 `db.sqlite3`를 열어보면, 우리가 만든 모델이 어떤 테이블로 변환되었는지 볼 수 있습니다. `django-debug-toolbar` 를 활용해보면, ORM 한 줄이 어떤 SQL로 번역되는지도 확인할 수 있습니다.

5.3 마이그레이션은 여러번 할 생각으로 임하기

모델을 잘못 짜도 괜찮습니다. `db.sqlite3`를 지우고 `migrations/` 안의 번호 파일들을 정리한 뒤 다시 `makemigrations` & `migrate` 하면 됩니다. **개발 단계에서 DB는 얼마든지 부수고 다시 만들어도 되는 것**입니다.

5.4 AI에게 SQL/모델을 시킬 때

> **AI에게 "게시판 좋아요 기능 만들어줘"라고 하면, AI는 거의 완벽한 코드를 뱉어낼 겁니다.** 그게 N:1로 짜졌는지 N:M으로 짜졌는지, `related_name`이 충돌하지 않는지, `on_delete`가 적절한지 — **그걸 검토할 수 있는 사람이 되는 것**이 진짜 목표입니다.

---

6. 학습 로드맵에서의 위치

```
[ Python · 알고리즘 · AI · Web · Django 기초 ]  <== 여기까지 완료
                        |
                        ▼
[ DB - SQL & SQLite (2일) ]                     <== 지금 시작
   · 데이터베이스가 뭔지, 왜 필요한지
   · SELECT / INSERT / UPDATE / DELETE / JOIN
                        |
                        ▼
[ DB - Django 모델 관계 (4일) ]
   · N:1 관계 (게시글-댓글, User-게시글)
   · N:M 관계 (좋아요, 팔로우)
                        |
                        ▼
[ DRF (Django REST Framework) ]
   · 모델 관계로 만든 데이터를 API로 노출
                        |
                        ▼
[ JavaScript / Vue ]
   · 그 API를 화면에서 받아서 보여주기
                        |
                        ▼
[ 최종 프로젝트 ]
   · 모델 설계 == 서비스 설계
```